

FITBUDDY

AI FITNESS PLAN GENERATOR

Project Documentation

PROJECT TEAM

Team Leader • T. Sanjay Team

Member • Mithesh J

Team Member • Naveen Kumar K

Team Member • SriKrishanth M

PROJECT TEAM

Team Leader • T. Sanjay • 47423E7ED197A2C9F0A4158D58D71875

Team Member • Mithesh J • 6B9280074C6C4BD917E40014F9341219

Team Member • Naveen Kumar K • 49C6EB16109FEACABCF4898810SFACTO Team

Member • SriKrishanth M • 17910853CBBD22285094C6728988D0C6

September 2026

— 1 —

Project Overview

A concise introduction to the FitBuddy system.

PROJECT VISION

FitBuddy combines a responsive web experience, Python backend processing, database functionality and Gemini generative AI to create a structured 7-day fitness blueprint from user-provided goals and information.

Project Identity

Component	Technology / Purpose
Project	FitBuddy
Category	AI-assisted fitness planning web application
Primary AI	Google Gemini
Backend	Python / FastAPI-based application structure
Frontend	HTML, CSS and JavaScript

1. Abstract

FitBuddy is an AI-powered web application designed to generate personalized fitness plans from user-provided fitness information and goals. The system collects inputs such as age, weight, fitness goal and preferred intensity, processes the information through a Python backend, and uses Google Gemini models to generate a structured 7-day fitness blueprint. The resulting plan is presented through a browser-based interface for easy review.

2. Introduction

Creating a suitable fitness routine can be difficult when users have different goals, measurements, preferences and activity levels. FitBuddy attempts to simplify this process by collecting relevant requirements and converting them into an organized fitness plan with generative AI assistance.

ACADEMIC VALUE

The project demonstrates frontend design, backend routing, database integration, API usage, testing and generative-AI integration in one practical application.

3. Problem Statement

Users may find it difficult to create a structured fitness routine that matches their personal information and fitness objective. Generic plans may not reflect individual requirements. FitBuddy addresses this by collecting user-specific inputs and generating an organized 7-day fitness blueprint through an AI-assisted workflow.

— 3 —

4. Objectives

- Provide a simple web interface for generating personalized fitness plans.
- Collect relevant user information and fitness preferences through forms.
- Generate a structured 7-day fitness blueprint using Gemini models.
- Process user requests through a Python backend.
- Support database-backed application functionality.
- Provide authentication and protected functionality where configured.
- Present generated recommendations in a clear and readable format.
- Demonstrate practical full-stack and generative-AI development.

5. Key Features

- AI-powered personalized fitness-plan generation.
- 7-day fitness blueprint generation.
- User input for age, weight, goal and intensity.
- Fitness-goal based planning support.
- Gemini AI integration.
- Python backend with API/web routing.
- Database layer for application data.
- Responsive browser-based interface.
- Testing support through a dedicated test module.

— 4 —

6. Technology Stack

Component	Technology / Purpose
Frontend	HTML, CSS and JavaScript — browser interface and user interaction
Backend	Python with FastAPI/web API architecture
AI Layer	Google Gemini generative-AI models
Database	Python application database layer
Testing	Python / pytest-based test support
Configuration	Environment variables for secrets and API credentials
Deployment	Suitable for Python web hosting such as Render

7. System Architecture

FitBuddy follows a client-server architecture. The browser provides the presentation layer. User inputs are sent to the Python backend, where routes validate and process requests. Application logic interacts with the database layer and Gemini service, and the generated fitness plan is returned to the interface.

DATA FLOW
USER → WEB INTERFACE → PYTHON / FASTAPI → ROUTES & LOGIC → DATABASE / GEMINI AI → FITNESS BLUEPRINT → USER

8. Module Description

User / Input Module — Collects the information required for fitness-plan generation, including personal and goal-related inputs.

Authentication Module — Handles application access and protected functionality where authentication is configured.

Fitness Plan Module — Processes fitness requirements and prepares requests for plan generation.

Gemini AI Service — Connects the backend to the configured Gemini model and converts inputs into generated plan content.

Database Module — Provides database connection and data-management functionality. **Routes**

& Schemas — Defines application endpoints and structured request/response data. **Testing**

Module — Provides automated test support for checking application behaviour.

9. Typical Workflow

1. User opens FitBuddy in a web browser.
2. User accesses the relevant page or authentication flow.
3. User enters fitness information and selects a goal and intensity.
4. Frontend submits the request to the Python backend.
5. Backend routes validate and process the data.
6. Application prepares a prompt for Gemini.
7. Gemini generates fitness-plan content.
8. Backend processes the response.
9. The 7-day blueprint is displayed to the user.

— 6 —

10. Project Structure

```
Fit_Buddy/
├── app/
│   ├── _init_.py
│   ├── database.py
│   ├── gemini_service.py
│   ├── main.py
│   ├── models.py
│   ├── routes.py
│   ├── schemas.py
│   ├── seed.py
│   └── utils.py
├── templates/
├── static/
├── tests/
├── project-documentation/
├── requirements.txt
├── .gitignore
├── README.md
├── run.bat
└── run.sh
```

GITHUB NOTE

Source files, README, requirements.txt, tests and documentation can be committed. Generated files such as `_pycache_`, `*.pyc` and `.pytest_cache`, plus API keys and passwords, should remain excluded.

11. Installation & Setup

10. Install Python 3.x.
11. Download or clone the FitBuddy repository.
12. Open a terminal in the project directory.
13. Create and activate a virtual environment.
14. Run: `pip install -r requirements.txt`
15. Configure Gemini API keys and other secrets through environment variables.
16. Start the FastAPI application using the configured run command.
17. Open the local URL in a browser.

— 7 —

12. Configuration & Security

- Gemini API keys and secret credentials should never be committed to GitHub.
- Use environment variables or a local .env configuration for secrets.
- Disable development/debug settings in production.
- Protect authentication credentials and securely handle passwords.
- Validate user-provided input before processing.
- Use HTTPS and suitable production database/storage configuration when deployed.

IMPORTANT

Keep `GOOGLE_API_KEY`, passwords, tokens and other secrets outside the public repository.

13. Testing

Testing should cover application startup, valid and invalid inputs, fitness-plan generation, backend routes, authentication-related flows where configured, Gemini-service handling and error conditions.

Component	Technology / Purpose
Valid input	Application accepts the request and returns the intended plan.
Invalid input	Application validates or rejects the input gracefully.
AI service	Generated content is returned when the configured service is available.
AI failure	Application handles unavailable/failed AI responses without exposing secrets.
Protected routes	Unauthorized access is handled according to the configured authentication flow.

14. Advantages

- Personalized fitness-plan generation from user inputs.
- AI-assisted creation of structured 7-day plans.
- Simple browser-based workflow.
- Modular Python backend structure.
- Can be extended with additional fitness goals and planning features.
- Demonstrates frontend, backend, database and generative-AI integration.

15. Limitations

- AI-generated plans depend on the configured AI service and its availability.
- Generated fitness content is informational and should not be treated as medical or professional advice.
- Recommendation quality depends on the accuracy and completeness of user inputs.
- Production use requires stronger security, validation, monitoring and scalable data storage.
- AI responses may require additional validation before being treated as final recommendations.

16. Future Enhancements

- Add more fitness goals and activity categories.
- Add progress tracking and workout history.
- Add nutrition and meal-planning features.
- Add downloadable/shareable fitness plans.
- Add feedback mechanisms for generated plans.
- Add richer user profiles and preference tracking.
- Move to a managed production database.
- Add stronger automated testing and CI/CD.

17. Conclusion

FitBuddy demonstrates how a Python web application can combine structured user inputs, backend processing, database functionality and generative AI to create a personalized fitness-planning experience. The system is designed around the generation of a 7-day fitness blueprint and provides a foundation that can be expanded with progress tracking, nutrition planning, richer personalization and production deployment.

18. Viva / Interview Questions

18. Why did you choose Python/FastAPI for the backend?
19. How does user input reach the backend?
20. How is Gemini integrated into FitBuddy?
21. What information is used to personalize the plan?
22. How do you protect the Gemini API key?
23. What is the purpose of the database module?
24. How is the 7-day fitness blueprint generated?
25. How do you test the application?
26. What happens if the Gemini service is unavailable?
27. How would you deploy FitBuddy for public access?
28. What security improvements are required for production?
29. What features can be added in future versions?

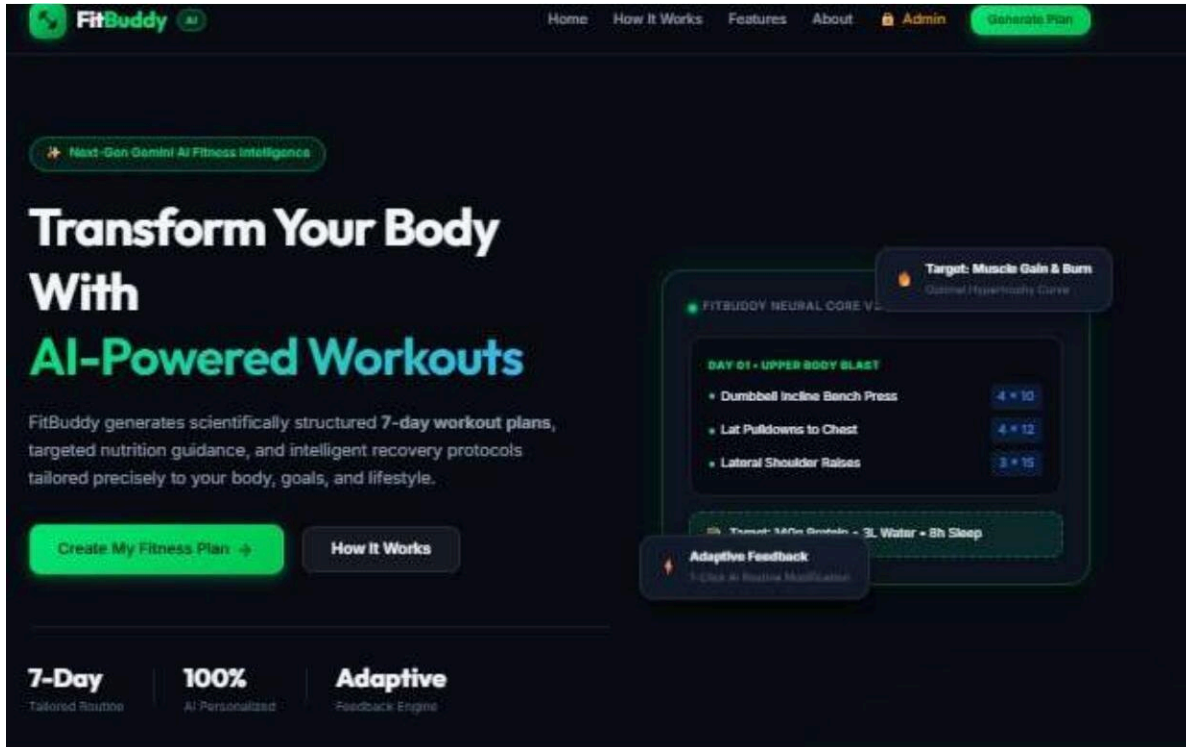
— 10 —

Appendix A. Application Screenshots

Screenshots of the actual FitBuddy application, arranged in the order a user experiences them.

A.1 Home / Landing Page

Main FitBuddy interface — the first screen users see.



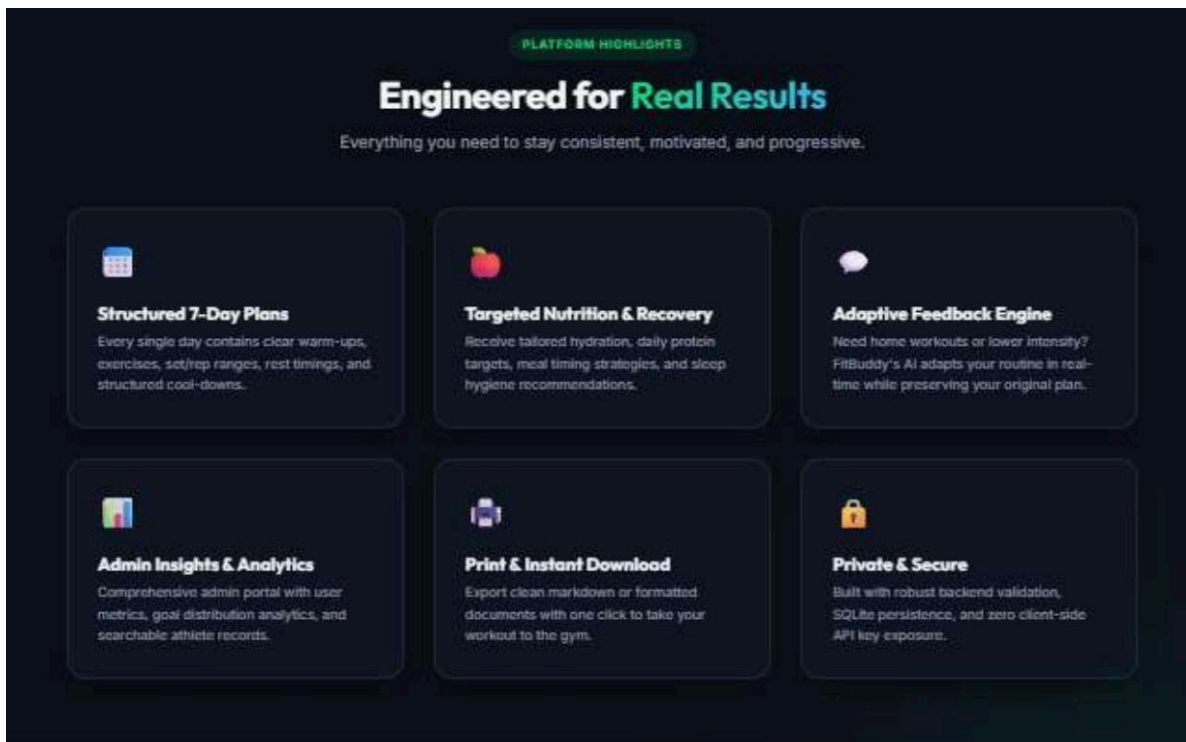
A.2 How FitBuddy Works

The 3-step process users follow to generate a plan.



A.3 Platform Features

Key platform highlights showcased on the FitBuddy landing page.



A.4 Fitness Input Form

Form for entering age, weight, goal and intensity.

Instant AI Generation

Build Your Personalized 7-Day Plan

Fill in your fitness parameters below to generate your custom AI workout & recovery routine.

Full Name * **Unique User ID ***

Age (Years) * **Weight (kg) ***

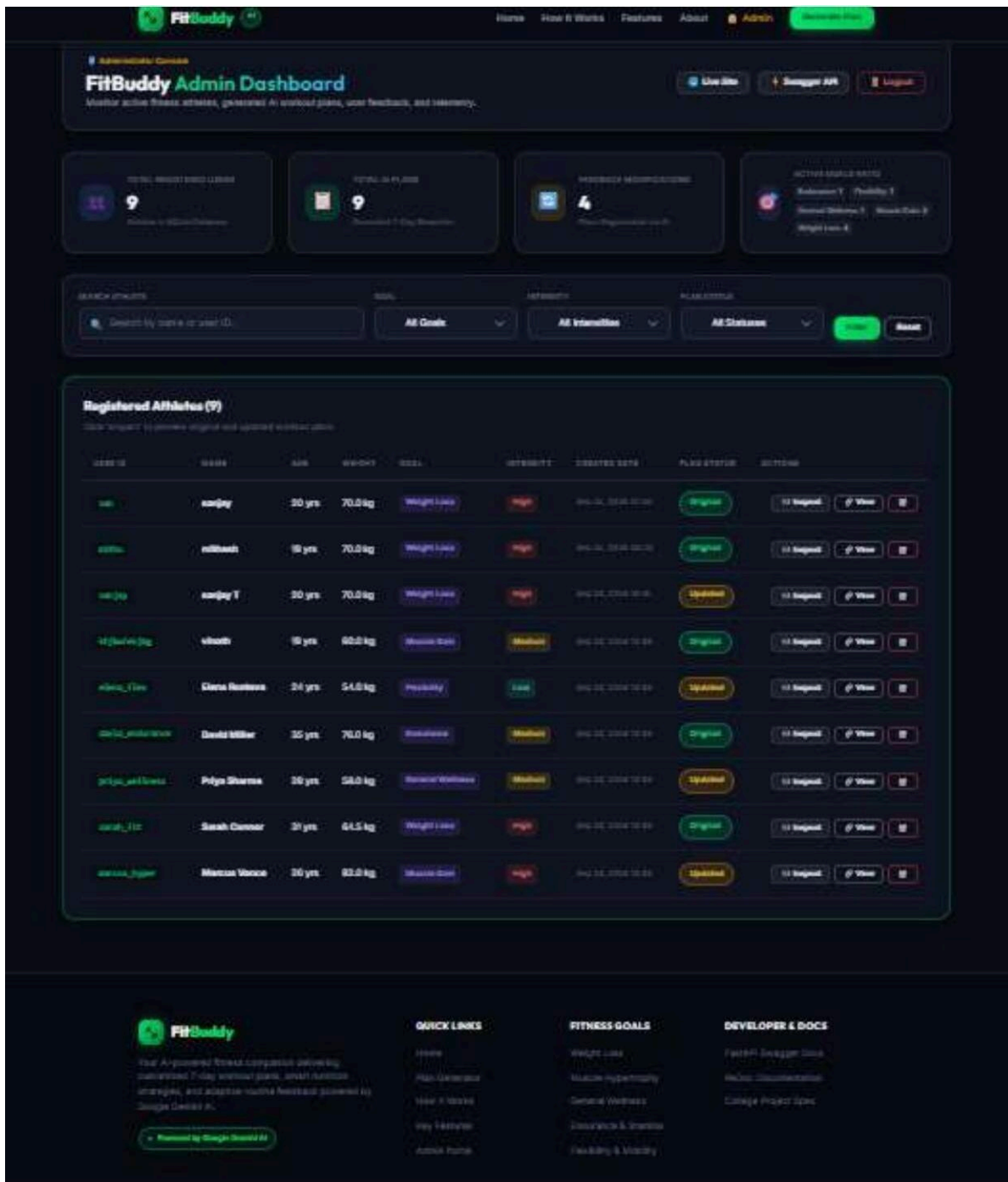
Fitness Goal * **Workout Intensity ***

All plans include safety checks, dynamic warm-ups, periodized rest days, and nutrition tips.

Generate My 7-Day Plan

A.5 Admin Dashboard & Generated Plans

Admin dashboard showing registered athletes, generated AI plan counts, and original/updated plan status.



The screenshot displays the FitBuddy Admin Dashboard. At the top, there's a navigation bar with links for Home, How It Works, Features, About, Admin, and a Logout button. The main header area includes the FitBuddy logo and a subtitle 'FitBuddy Admin Dashboard' with a description: 'Monitor active fitness athletes, generated AI workout plans, user feedbacks, and telemetry.' Below this, there are four summary cards: 'TOTAL REGISTERED ATHLETES' (9), 'TOTAL AI PLANS' (9), 'FITBUDDY MEMBERSHIPS' (4), and 'ACTIVE GYM CARD' (1). A search bar and filters for 'All Goals', 'All Intensity', and 'All Statuses' are present. The main section is titled 'Registered Athletes (9)' and contains a table with columns for ID, Name, Age, Weight, Goal, Intensity, Created Date, Plan Status, and Actions. The table lists 9 athletes with their respective details and plan statuses (Original or Updated). The footer section includes the FitBuddy logo, a description of the AI-powered fitness plan generator, and four columns of links: Quick Links, Fitness Goals, and Developer & Docs.

ID	Name	Age	Weight	Goal	Intensity	Created Date	Plan Status	Actions
1001	John Doe	30 yrs	70.0 kg	Weight Loss	High	2024-10-26 10:30	Original	10 Segments, View, Edit
1002	Jane Smith	28 yrs	65.0 kg	Weight Loss	High	2024-10-26 10:30	Original	10 Segments, View, Edit
1003	Mike Johnson	32 yrs	75.0 kg	Weight Loss	High	2024-10-26 10:30	Updated	10 Segments, View, Edit
1004	Sarah Lee	25 yrs	60.0 kg	Muscle Gain	Medium	2024-10-26 10:30	Original	10 Segments, View, Edit
1005	David Brown	35 yrs	80.0 kg	Flexibility	Low	2024-10-26 10:30	Updated	10 Segments, View, Edit
1006	Emily White	27 yrs	68.0 kg	Endurance	Medium	2024-10-26 10:30	Original	10 Segments, View, Edit
1007	Chris Garcia	31 yrs	72.0 kg	Muscle Gain	Medium	2024-10-26 10:30	Updated	10 Segments, View, Edit
1008	Alex Taylor	29 yrs	70.0 kg	Weight Loss	High	2024-10-26 10:30	Original	10 Segments, View, Edit
1009	Maria Lopez	26 yrs	62.0 kg	Muscle Gain	High	2024-10-26 10:30	Updated	10 Segments, View, Edit

A.6 Technology Stack Highlights

Technology stack and academic architecture summary shown on the site.

The screenshot displays the FitBuddy website's technology stack highlights and footer. The main section, titled "ACADEMIC & TECHNICAL ARCHITECTURE", features the heading "Built with Modern Full-Stack Technologies". Below this, a paragraph states: "FitBuddy was developed as an advanced AI web application integrating FastAPI, SQLite (SQLAlchemy ORM), and Google Gemini AI to deliver personalized physical conditioning and wellness solutions." A grid of technology tags includes Python 3.11, FastAPI, Google GenAI / Gemini, SQLAlchemy ORM, SQLite, Jinja2, VueJS C553, and Modern ES6 JS. To the right, three performance metrics are listed: 100% Zero Frontend API Key Exposure, < 3s AI Generation Latency, and Dual Original & Modified Plan Persistence. The footer contains the FitBuddy logo, a description of the app as an AI-powered fitness companion, a "Powered by Google Gemini AI" badge, quick links (Home, Plan Generator, How It Works, Key Features, Admin Portal), fitness goals (Weight Loss, Muscle Hypertrophy, General Wellness, Endurance & Stamina, Flexibility & Mobility), and developer resources (FastAPI Swagger Docs, ReDoc Documentation, College Project Spec). Copyright information for 2026 and a disclaimer for academic demonstration are also present.

ACADEMIC & TECHNICAL ARCHITECTURE

Built with Modern Full-Stack Technologies

FitBuddy was developed as an advanced AI web application integrating FastAPI, SQLite (SQLAlchemy ORM), and Google Gemini AI to deliver personalized physical conditioning and wellness solutions.

Python 3.11 FastAPI Google GenAI / Gemini SQLAlchemy ORM SQLite Jinja2 VueJS C553 Modern ES6 JS

100%
Zero Frontend API Key Exposure

< 3s
AI Generation Latency

Dual
Original & Modified Plan Persistence

FitBuddy
Your AI-powered fitness companion delivering customized 7-day workout plans, smart nutrition strategies, and adaptive routine feedback powered by Google Gemini AI.

Powered by Google Gemini AI

QUICK LINKS
[Home](#)
[Plan Generator](#)
[How It Works](#)
[Key Features](#)
[Admin Portal](#)

FITNESS GOALS
[Weight Loss](#)
[Muscle Hypertrophy](#)
[General Wellness](#)
[Endurance & Stamina](#)
[Flexibility & Mobility](#)

DEVELOPER & DOCS
[FastAPI Swagger Docs](#)
[ReDoc Documentation](#)
[College Project Spec](#)

© 2026 FitBuddy. Built with FastAPI, SQLite, and Google Gemini AI. For academic demonstration and general wellness guidance only.

A.7 Development Environment

FitBuddy source structure and development environment.

```
html lang="en" >
body>

  <div class="glow-orb glow-orb-3"></div>

  <!-- Navigation Bar -->
  <header class="navbar-wrapper">
    <nav class="navbar container">
      <a href="/" class="brand-logo">
        <div class="brand-icon">
          <svg viewBox="0 0 24 24" width="24" height="24" fill="currentColor">
            <path d="M20.57 14.86L22 13.43 20.57 12 17 15.57 8.43 7 12 3.43 10.57 2 9.14" />
          </svg>
        </div>
        <span class="brand-text">Fit<span class="brand-accent">Buddy</span></span>
        <span class="ai-badge">AI</span>
      </a>

      <!-- Desktop Navigation Links -->
      <ul class="nav-links" id="navLinks">
        <li><a href="/" class="nav-link">Home</a></li>
        <li><a href="/#how-it-works" class="nav-link">How It Works</a></li>
        <li><a href="/#features" class="nav-link">Features</a></li>
        <li><a href="/#about" class="nav-link">About</a></li>
        <li><a href="/view-all-users" class="nav-link admin-nav-link"><span class="lock-icon" /></a></li>
        <li><a href="/#generator" class="btn btn-sm btn-primary nav-cta">Generate Plan</a></li>
      </ul>
    </nav>
  </header>
```

Appendix B. Generated Fitness Blueprint Example

Example project output documented during development: a personalized 7-day weight-loss fitness blueprint generated for a profile with age 20 years, weight 70.0 kg, goal Weight Loss and High intensity. This example is included as a project-output illustration rather than professional fitness advice.

Component	Technology / Purpose
Plan Title	7-Day Weight Loss AI Fitness Blueprint
Age	20 years
Weight	70.0 kg
Goal	Weight Loss
Intensity	High
Duration	7 Days

Appendix D. GitHub Repository Checklist

Repository Link: https://github.com/sanjayt8610-sys/Fit_Buddy

- Application source code (.py files) — included.
- HTML/CSS/JavaScript frontend — included.
- Tests and project documentation — included.
- requirements.txt — included.
- .gitignore — included.
- README.md — included.
- run.bat and run.sh — included.
- _pycache_, *.pyc and .pytest_cache — excluded.
- API keys, passwords and secret credentials — excluded.

FINAL PROJECT

FITBUDDY — AI FITNESS PLAN GENERATOR

A full-stack academic project combining web development, Python backend processing, database functionality and Gemini generative AI.